



UAI301 Object Oriented Programming

Lab Assignment 5.2

Name: Amritanshu Mishra

Roll No.: 1025250008

Sub-Group: 2X11

Subject Code: UAI301

Subject Name: Object Oriented Programming

Question 1 Case Study 1: FinTech & Banking (Account Security). Create a BankAccount class with private **balance**, public **deposit(amount)**, **withdraw(amount)**, and **get_balance()**. Attempt to directly overwrite **account.balance** and verify via **get_balance()** that it remained untampered.

```
In [1]: class BankAccount:
    def __init__(self, owner, initial_balance):
        self.owner = owner
        self.__balance = initial_balance

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount
            print(f"Deposited {amount}. Balance: {self.__balance}")
        else:
            print("Deposit amount must be greater than 0.")

    def withdraw(self, amount):
        if amount <= self.__balance:
            self.__balance -= amount
            print(f"Withdrew {amount}. Balance: {self.__balance}")
        else:
            print("Insufficient balance.")

    def get_balance(self):
        return self.__balance

account = BankAccount("Aman", 5000)
account.deposit(2000)
account.withdraw(1000)

account.__balance = 1000000
print("Attempted direct overwrite: account.__balance = 1000000")
print("Actual balance via get_balance():", account.get_balance())
```

Deposited 2000. Balance: 7000
Withdrew 1000. Balance: 6000
Attempted direct overwrite: account.__balance = 1000000
Actual balance via get_balance(): 6000

Question 2 Case Study 2: E-Commerce Order Processing (Protected Logistics). Create a parent class Order with protected `_shipping_status` and a subclass ExpressOrder with `expedite_shipping()` and `track_order()` to demonstrate protected members in inheritance.

```
In [2]: class Order:
        def __init__(self):
            self._shipping_status = "Processing"

        class ExpressOrder(Order):
            def expedite_shipping(self):
                self._shipping_status = "In Transit - Priority"

            def track_order(self):
                return self._shipping_status

order1 = ExpressOrder()
print("Initial Status:", order1.track_order())
order1.expedite_shipping()
print("Updated Status:", order1.track_order())
```

Initial Status: Processing
Updated Status: In Transit - Priority

Question 3 Case Study 3: Healthcare Records & Patient Data (Flexible Getters/Setters). Create a Patient class with private **name and age**, and custom access methods `fetch_age()` and `update_age(new_age)` that validate age is between 0 and 120, testing both a valid (35) and invalid (150) update.

```
In [3]: class Patient:
        def __init__(self, name, age):
            self.__name = name
            self.__age = age

        def fetch_age(self):
            return self.__age

        def update_age(self, new_age):
            if 0 <= new_age <= 120:
                self.__age = new_age
                print(f"Age updated to {new_age}.")
            else:
                print("Invalid age. Update cancelled.")

p1 = Patient("Rahul Sharma", 30)
print("Current Age:", p1.fetch_age())
```

```

p1.update_age(35)
print("Age after valid update:", p1.fetch_age())

p1.update_age(150)
print("Age after invalid update:", p1.fetch_age())

```

Current Age: 30
 Age updated to 35.
 Age after valid update: 35
 Invalid age. Update cancelled.
 Age after invalid update: 35

Question 4 Case Study 4: Enterprise Payroll System (Preventing Subclass Overwrites). Create a base class `BasePayroll` with private **`tax_rate = 0.20`** and **`calculate_base_tax(salary)`**, and a child class **`RegionalPayroll`** that **sets** `tax_rate = 0.05`, demonstrating that name mangling keeps the two variables completely separate.

```

In [4]: class BasePayroll:
        def __init__(self):
            self.__tax_rate = 0.20

        def calculate_base_tax(self, salary):
            return salary * self.__tax_rate

        class RegionalPayroll(BasePayroll):
            def __init__(self):
                super().__init__()
                self.__tax_rate = 0.05

        payroll = RegionalPayroll()
        print("Base tax on salary 100000:", payroll.calculate_base_tax(100000))
        print("This proves calculate_base_tax() still uses the parent's tax rate")
        print("because name mangling keeps '_BasePayroll__tax_rate' and '_RegionalPayroll__tax_rate' separate.")

```

Base tax on salary 100000: 20000.0
 This proves `calculate_base_tax()` still uses the parent's tax rate (0.20), because name mangling keeps `'_BasePayroll__tax_rate'` and `'_RegionalPayroll__tax_rate'` separate.

Question 5 Case Study 5: Smart Home IoT Device (Controlled State Changes). Create a `Thermostat` class with private `__target_temp` defaulting to 22, `set_temperature(temp)` validating `16 <= temp <= 30`, and `get_temperature()`. Demonstrate attempting to set the temperature to 10°C and 24°C.

```

In [5]: class Thermostat:
        def __init__(self):
            self.__target_temp = 22

        def set_temperature(self, temp):

```

```

    if 16 <= temp <= 30:
        self.__target_temp = temp
        print(f"Temperature set to {temp}°C.")
    else:
        print(f"Warning: {temp}°C is outside the safe range (16-30°C)")

    def get_temperature(self):
        return self.__target_temp

thermo = Thermostat()
print("Initial Temperature:", thermo.get_temperature())

thermo.set_temperature(10)
print("Current Temperature:", thermo.get_temperature())

thermo.set_temperature(24)
print("Current Temperature:", thermo.get_temperature())

```

Initial Temperature: 22
Warning: 10°C is outside the safe range (16-30°C). Temperature not changed.
Current Temperature: 22
Temperature set to 24°C.
Current Temperature: 24

Question 6 Case Study 6: Corporate HR Portal (Employee Directory Access Levels). Create an Employee class with public name, protected `_project`, and private `salary`, and a child class `Developer` with `get_dev_summary()`. Access name and `_project` directly, handle the `AttributeError` from accessing `salary` directly, and extract `__salary` via name mangling.

```

In [6]: class Employee:
        def __init__(self, name, project, salary):
            self.name = name
            self._project = project
            self.__salary = salary

        class Developer(Employee):
            def get_dev_summary(self):
                return f"{self.name} is working on {self._project}"

dev1 = Developer("Aman", "AI Platform", 90000)

print("Name:", dev1.name)
print("Project:", dev1._project)

try:
    print(dev1.__salary)
except AttributeError as e:
    print("Error accessing __salary directly:", e)

print("Salary via name mangling:", dev1._Employee__salary)
print(dev1.get_dev_summary())

```

Name: Aman
Project: AI Platform
Error accessing __salary directly: 'Developer' object has no attribute '__salary'
Salary via name mangling: 90000
Aman is working on AI Platform

Question 7 Case Study 7: Digital Wallet Service (Pin-Protected Transactions). Create a DigitalWallet class with private **balance** and **pin**, implement **verify_pin(entered_pin)** and **transfer_funds(amount, target_wallet, entered_pin)**, and test both a successful transfer and a failed transfer with an incorrect PIN.

```
In [7]: class DigitalWallet:
        def __init__(self, balance, pin):
            self.__balance = balance
            self.__pin = pin

        def verify_pin(self, entered_pin):
            return entered_pin == self.__pin

        def transfer_funds(self, amount, target_wallet, entered_pin):
            if not self.verify_pin(entered_pin):
                print("Transfer failed: Incorrect PIN.")
                return
            if self.__balance < amount:
                print("Transfer failed: Insufficient balance.")
                return
            self.__balance -= amount
            target_wallet.__balance += amount
            print(f"Transferred {amount} successfully.")

        wallet1 = DigitalWallet(5000, "1234")
        wallet2 = DigitalWallet(1000, "5678")

        wallet1.transfer_funds(1000, wallet2, "1234")
        wallet1.transfer_funds(1000, wallet2, "0000")
```

Transferred 1000 successfully.
Transfer failed: Incorrect PIN.

Question 8 Case Study 8: University Grading System (Read-Only Transcripts). Create a StudentTranscript class with public **student_id** and private **__grades** dictionary, implement **add_grade(subject, grade)**, **calculate_gpa()**, and a read-only **get_transcript()** that returns a copy so external users cannot mutate internal state.

```
In [8]: class StudentTranscript:
        def __init__(self, student_id):
            self.student_id = student_id
            self.__grades = {}
```

```

def add_grade(self, subject, grade):
    self.__grades[subject] = grade

def calculate_gpa(self):
    if not self.__grades:
        return 0
    return sum(self.__grades.values()) / len(self.__grades)

def get_transcript(self):
    return dict(self.__grades)

t1 = StudentTranscript("102403001")
t1.add_grade("Maths", 9)
t1.add_grade("Physics", 8)
t1.add_grade("OOP", 9)

print("GPA:", t1.calculate_gpa())

transcript = t1.get_transcript()
transcript["Maths"] = 2
print("External copy modified, but internal record is safe:")
print("get_transcript() again:", t1.get_transcript())

```

GPA: 8.666666666666666

External copy modified, but internal record is safe:

get_transcript() again: {'Maths': 9, 'Physics': 8, 'OOP': 9}